

Examen de la primera convocatoria (13/06/2022)

- Leer completamente cada ejercicio antes de empezar a trabajar.
- Resolver los ejercicios en **hojas separadas**.
- Si no da tiempo a resolver alguna parte, se tendrá en cuenta el indicar en *detalle* cómo se resolvería. Hacer un procedimiento o función que resuelve el problema **no** es resolver el ejercicio: se trata de **incidir en los aspectos de especificación y diseño**.
- Si algo no se especifica explícitamente, indicar las decisiones adoptadas (ej.: “como no se indica lo contrario, supongo que ...”).
- **Comentar y anotar** los programas adecuadamente.
- No se admitirán soluciones a lápiz ni con bolígrafo rojo.
- Todas las hojas deben llevar apellidos, nombre y número de orden.
- Duración máxima del examen: **3 horas**

Ej. 1 — (3 puntos) Para celebrar el fin de curso, tú y tus compañeros de Algoritmia Básica habéis organizado una quedada en el parque de la Expo. Has pensado que podría ser divertido jugar a un “tira y afloja” después de la comida y bebida. Para el tira y afloja, tenéis que dividirlos en dos equipos tan equilibrados como sea posible; en particular, el número de personas en los dos equipos no debe diferir en más de uno y el peso total de las personas en cada equipo debe ser lo más igual posible. Considera que sois n participantes y conoces el peso p_i de cada participante i ($i = 1, \dots, n$). Diseña un algoritmo de ramificación y poda para formar los dos equipos considerando las mencionadas restricciones. Para ello define:

- a) Una representación para las soluciones y describe el árbol del espacio de soluciones.
- b) La función de coste $c(x)$, para cada nodo x del árbol.
- c) Una función de estimación del coste $\hat{c}(x)$ para la ramificación.
- d) Una función de poda, $\mathcal{U}(x)$.

Ej. 2 — (2 puntos) Se está celebrando una subasta para vender n objetos. Se reciben m pujas, cada una de la forma ‘quiero k_i objetos por e_i euros’ ($i = 1 \dots m$). Caracterizar el problema como un problema de la mochila. ¿En qué condiciones es una mochila 0-1 o fraccional?

Ej. 3 — (2 puntos) Juan ha recibido dinero de sus padres esta semana y quiere gastarlo todo comprando libros. Terminar un libro le cuesta una semana, porque le gusta disfrutar cada palabra mientras lee. Puesto que Juan recibe dinero cada dos semanas, ha decidido comprar dos libros, luego puede leerlos hasta recibir más dinero. Quiere gastar los d euros recibidos y tiene que elegir dos libros cuyos precios sumados sean iguales a d . La lista de preferencias de Juan está ordenada según el precio, de menor a mayor. Es un poco difícil encontrar estos libros (su lista de preferencias es muy larga): diseña un algoritmo con coste en tiempo en $O(n \log n)$ para encontrarlos.

Ej. 4 — (3 puntos) Se denomina *arbitraje* al uso de las discrepancias en las tasas de cambio de divisas para obtener un beneficio. Por ejemplo, durante un corto espacio de tiempo puede suceder que un euro valga 0'95 dólares, un dólar valga 0'75 libras esterlinas y una libra esterlina valga 1'45 euros. En una situación como esa, partiendo de un euro podemos obtener:

$$1 \text{ €} = 1 \cdot 0'95 (\$/\text{€}) \cdot 0'75 (\text{£}/\$) \cdot 1'45 (\text{£}/\text{€}) = 1'033125 \text{ euros}$$

con un beneficio neto de algo más del 3'3 %.

Supondremos que hay n tipos diferentes de moneda, y que se dispone de una tabla $C(1..n, 1..n)$ donde $C(i, j)$ es la tasa de cambio entre (una unidad de) la divisa i y la divisa j . Por ejemplo, si el euro es la divisa 1, y la libra esterlina la 2, entonces (según hemos dicho antes) $C(2, 1) = 1'45$. Se cumple que $C(i, j) = 1/C(j, i)$ para todo i y j , y que $C(i, i) = 1$.

Diseñar un algoritmo de programación dinámica que dada la matriz C y el identificador i de una divisa, nos proporcione el valor del mejor arbitraje, donde este consiste en una secuencia $i_1, i_2, \dots, i_k$ de identificadores distintos entre si, y distintos de i , tal que su valor $C(i, i_1) \cdot C(i_1, i_2) \cdot \dots \cdot C(i_{k-1}, i_k) \cdot C(i_k, i)$ es máximo.

Examen de la segunda convocatoria (13/09/2022)

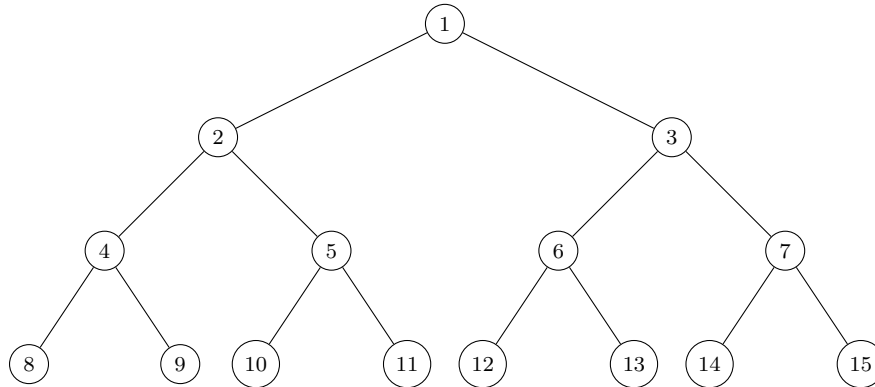
- Leer completamente cada ejercicio antes de empezar a trabajar.
- Resolver los ejercicios en **hojas separadas**.
- Si no da tiempo a resolver alguna parte, se tendrá en cuenta el indicar en *detalle* cómo se resolvería. Hacer un procedimiento o función que resuelve el problema **no** es resolver el ejercicio: se trata de **incidir en los aspectos de especificación y diseño**.
- Si algo no se especifica explícitamente, indicar las decisiones adoptadas (ej.: “como no se indica lo contrario, supongo que ...”).
- **Comentar** y **anotar** los programas adecuadamente.
- No se admitirán soluciones a lápiz ni con bolígrafo rojo.
- Todas las hojas deben llevar apellidos, nombre y número de orden.
- Duración máxima del examen: **3 horas**

Ej. 1 — (2,5 puntos) Para sensibilizar los niños sobre la protección de los animales, en un colegio se ha decidido que cada niño apadrine un animal mediante una adopción simbólica de una especie diferente. Hay n niños en el colegio: cada niño ha dado su preferencia para cada una de las n especies diferentes que se pretenden apadrinar (es decir, 0 no me gusta, 1 me gusta). Diseña un algoritmo de ramificación y poda que determine la asignación niño-animal que maximice la satisfacción global. Esta se calcula sumando las preferencias dadas por los niños por apadrinar una especie. Para ello, considera el esquema de ramificación y poda de **mínimo coste** (tal como se ha visto en los ejemplos en clase) y define:

- a)** Una representación para las soluciones, precisando las restricciones explícitas e implícitas, y el árbol del espacio de soluciones.
- b)** La función de coste $c(x)$, para cada nodo x del árbol, que hay que minimizar.
- c)** Una función de estimación adecuada, $\hat{c}(x)$, para ser utilizada como función de prioridad de los nodos vivos.
- d)** Una función de poda adecuada, $\mathcal{U}(x)$.

Ej. 2 — (2,5 puntos) Como si fuera una máquina de pinball se dejan caer una cantidad de n bolas, una por una, desde la raíz de una estructura de árbol binario completo. Todos los nodos del árbol están numerados de forma secuencial, comenzando en 1 con la raíz (nivel 1), y luego los hijos de nivel 2, y así sucesivamente. Los nodos en cualquier nivel están numerados de izquierda a derecha. Cada vez que una bola se deja caer, visita primero un nodo no terminal del árbol, continúa moviéndose hacia abajo, ya sea siguiendo el camino del sub-árbol izquierdo o el camino del sub-árbol derecho, hasta que llega en uno de los nodos hoja del árbol. Para determinar la dirección de movimiento de una bola, cada nodo no terminal tiene asociado un valor booleano (*flag*): falso o verdadero. Al comienzo del juego, todos los *flag* tienen valor falso. Al visitar un nodo no terminal, si el valor actual del *flag* en este nodo es falso, la bola primero cambiará el

valor del nodo, es decir, de falso a verdadero, y luego seguirá el sub-árbol izquierdo de este nodo para seguir moviéndose hacia abajo. De lo contrario, también cambiará el valor del *flag*, es decir, de verdadero a falso, pero seguirá el sub-árbol derecho de este nodo para seguir moviéndose hacia abajo. Por ejemplo, la figura a continuación representa un árbol binario completo de nivel 4.



Puesto que todos los *flag* están configurados inicialmente como falsos, la primera bola que se deja caer cambiará los valores de los *flag* en los nodos 1, 2 y 4 antes de que finalmente se detenga en la posición 8. La segunda bola que se suelta cambiará los valores de los *flag* en los nodos 1, 3 y 6, y se detendrá en la posición 12. La tercera bola que se deja caer cambiará los valores de los *flag* en los nodos 1, 2 y 5 antes de que se detenga en posición 10.

Diseña un algoritmo *divide y vencerás* que tome en entrada: 1) la profundidad máxima del árbol P y 2) la i -ésima bola, y determine la posición de parada de la bola. Puedes asumir que el valor i es menor o igual al número total de hojas del árbol.

Ej. 3 — (2,5 puntos) El profesor Sopmac quiere decorar el pasillo de su casa con cuadros. Como supone que sus visitantes se fijarán mejor en ellos si se encuentran próximos a las puertas, ha marcado en una recta las posiciones de las mismas.

Calcula que un cuadro está bien colocado si se encuentra a una distancia menor o igual de 0,75 metros de una puerta, y además piensa que cada una debería tener cerca un cuadro.

Lo que quiere es saber dónde colocar los cuadros de manera que se minimice el número que necesita. Para ello, debemos diseñar un algoritmo que encuentre una solución óptima.

Ej. 4 — (2,5 puntos) Dados N objetos de longitud $l_1, l_2, \dots, l_N$, (naturales positivos) y un número natural L , se desea saber si existe algún subconjunto de los N objetos cuya suma de longitudes sea exactamente L .

1. Diseñar una ecuación en recurrencias que devuelva verdad si existe tal subconjunto y falso en caso contrario.
2. Diseñar un algoritmo iterativo para la ecuación anterior con un coste en espacio de $O(L)$. Calcular el coste en tiempo.

Indicar cómo se modificaría el algoritmo anterior para que adicionalmente se obtenga uno de esos subconjuntos de objetos (en caso de que exista alguno). Calcular el coste en tiempo y en espacio de la solución.

ALGORITMIA BÁSICA
Grado en Ingeniería Informática y Doble Grado en Matemáticas-Ingeniería Informática
Examen de la primera convocatoria (31/05/2024)

- Resolver los ejercicios en **hojas separadas**.
- Si no da tiempo a resolver alguna parte, se tendrá en cuenta el indicar en *detalle* cómo se resolvería. Hacer un procedimiento o función que resuelve el problema **no** es resolver el ejercicio: se trata de **incidir en los aspectos de especificación y diseño**.
- Si algo no se especifica explícitamente, indicar las decisiones adoptadas (ej.: “como no se indica lo contrario, supongo que ...”).
- **Comentar y anotar** los algoritmos adecuadamente.
- No se admitirán soluciones a lápiz ni con bolígrafo rojo.
- Todas las hojas deben llevar apellidos, nombre y número de orden.
- Duración máxima del examen: **3 horas**

Ej. 1 — (2,5 puntos) Dado n escribir el máximo número de números compuestos (números que no son primos) que suman n (recordar que los primeros números compuestos son: 4, 6, 8, 9, 10, 12, 14, 15, 16, 18, 20, ...).

Ejemplos:

- Entrada: 90. Salida: 22

$$90 = 21 * 4 + 6$$

Observación: el 4 aparece 21 veces, se repite tantas como sea posible/necesario.

- Entrada: 10. Salida: 2

$$10 = 4 + 6$$

- Entrada: 29. Salida: 6

$$29 = 4 * 5 + 9$$

- Entrada: 35. Salida: 7

$$35 = 4 * 5 + 6 + 9$$

Notas:

1. Si el número es menor que 4 no habrá ninguna combinación.

2. Si el número es 5, 7, 11 no habrá ninguna división posible.

¿Es posible diseñar un algoritmo voraz para calcular la descomposición para números mayores o iguales que 4 que no sean 5, 7, 11?

Sugerencia: puesto que lo mejor es poner tantos 4's como sea posible, el resto de dividir el número por 4 nos ayudará a determinar qué número compuesto podemos usar para obtener la solución.

Ej. 2 — (2,5 puntos) Dada una lista $L[1..n]$ de n números enteros, queremos encontrar la longitud de la máxima sublista no decreciente (MSND), esto es, la longitud de la sublista más larga en la que sus elementos están en orden no decreciente.

Por ejemplo, para la lista $L = [3, 10, 2, 1, 20]$ las posibles sublistas no decrecientes son:

- Longitud 1: $[3], [10], [2], [1], [20]$

- Longitud 2: [3, 10], [3, 20], [10, 20], [2, 20], [1, 20]
- Longitud 3: [3, 10, 20]

No hay ninguna MSND de longitud mayor que 3.

Ejemplos:

- Entrada: [3, 10, 2, 1, 20]
Salida: 3
Explicación: la MSND es [3, 10, 20]
- Entrada: [3, 2]
Salida: 1
Explicación: hay dos MSND, y son [3] y [2]
- Entrada: [50, 3, 10, 7, 40, 80]
Salida: 4
Explicación: la MSND es [3, 7, 40, 80]

Puede demostrarse fácilmente que una MSND de una lista dada, L , contiene una MSND de un prefijo de L . Apoyándose en ese hecho:

1. Escribir la ecuación en recurrencias que representa la longitud de la MSND.
2. Diseñar un algoritmo de programación dinámica que resuelva el problema del cálculo de una MSND de una lista dada.
3. Analizar la eficiencia de la solución propuesta. No hace falta un análisis formal, basta con una idea genérica del coste.
4. Proponer una solución eficiente para resolver el problema.

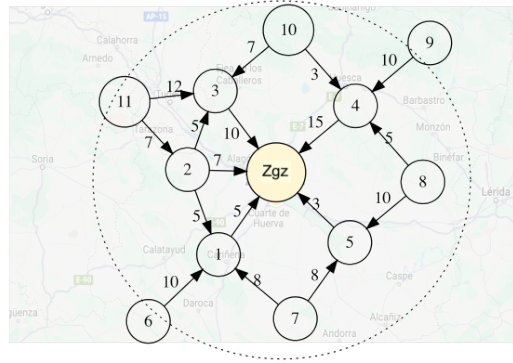
Ej. 3 — (2,5 puntos) Considera una lista $L[1..n]$ de n números enteros. Se define *inversión* como un par de índices (i, j) de la lista, con $i < j$ y $L[i] > L[j]$. Por ejemplo, en la lista [3, 1, 4, 1, 5, 9, 2] hay 7 inversiones: (1, 2), (1, 4), (1, 7), (3, 4), (3, 7), (5, 7) y (6, 7).

- a) ¿Cuál es el máximo número de inversiones que podemos encontrar en una lista de longitud n ? Justifica la respuesta.
- b) Describe un algoritmo *eficiente* para calcular el número total de inversiones en la lista $L[1..n]$.
- c) Calcula el coste asintótico en tiempo del algoritmo.

Sugerencia: Se puede pensar adaptar un algoritmo de ordenación de coste $O(n \log n)$.

Ej. 4 — (2,5 puntos) Has sido contratado recientemente por el CNPIC (Centro Nacional de Protección de Infraestructuras y Ciberseguridad) y trabajas en un equipo que se encarga de la protección de las infraestructuras viarias. Los terroristas que quieren atacar una ciudad pueden utilizar las carreteras que conducen a esa ciudad para el transporte de personal, armas u otros materiales peligrosos. Una contramedida para prevenir los ataques es desplegar sensores en las carreteras en un área circular, con centro la ciudad y un radio predefinido, con el objetivo de detectar el movimiento de los terroristas y bloquearlos antes de que lleguen a destino. Los sensores se pueden colocar a lo largo de cualquier segmento (i, j) delimitado por los cruces i y j de carreteras en el área. El número de sensores a colocar depende de la longitud l_{ij} del segmento (i, j) y, si se decide colocar sensores en un segmento, estos se tienen que colocar a una distancia de 0,5 km el uno del otro. Por ejemplo, si se decide colocar sensores en el segmento $(4, Zgz)$ entonces habrá que colocar $15/0,5 = 30$ sensores.

La figura muestra un ejemplo, donde los cruces y segmentos se representan como nodos y aristas, respectivamente, de un grafo dirigido. Los números asociados a las aristas indican la longitud l (en km) de los segmentos. Tienes que identificar los segmentos donde colocar los sensores para detectar los vehículos sospechosos que lleguen a la ciudad (*Zgz*) **utilizando el mínimo número de sensores**.



Sugerencia: Considera la relación que existe entre el máximo flujo y el mínimo corte.

- Define un problema de optimización para calcular el número mínimo de sensores necesarios.
- Resuelve el problema para el ejemplo indicado en la figura.
- En el ejemplo, indica los segmentos donde colocar los sensores. Justifica la respuesta.

ALGORITMIA BÁSICA
Grado en Ingeniería Informática y Doble Grado en Matemáticas-Ingeniería Informática
Examen extraordinario (25/06/2024)

- Resolver los ejercicios en **hojas separadas**. Todas las hojas deben llevar apellidos, nombre y número de orden.
- Si no da tiempo a resolver alguna parte, se tendrá en cuenta el indicar en *detalle* cómo se resolvería. Hacer un procedimiento o función que resuelve el problema **no** es resolver el ejercicio: se trata de **incidir en los aspectos de especificación y diseño**.
- Si algo no se especifica explícitamente, indicar las decisiones adoptadas (ej.: “como no se indica lo contrario, supongo que ...”).
- **Comentar** los algoritmos adecuadamente.
- No se admitirán soluciones a lápiz ni con bolígrafo rojo.
- Duración máxima del examen: **3 horas**

Ej. 1 — (2,5 puntos) Tenemos una pared de longitud L y suficientes estanterías para colgar de dos tipos de longitud $l_1, l_2 \leq L$ ($l_1 \neq l_2$); las más largas son más baratas. El objetivo es poner estanterías en la pared de forma que se minimice la longitud que queda libre y, en caso de soluciones con la misma longitud libre, preferimos las más baratas.

Ejemplos:

Entrada	Salida	Comentarios
$L = 24, l_1 = 3, l_2 = 5$	3 3 0	Tres unidades de cada tipo y quedará 0 espacio libre. $3 \cdot 3 + 3 \cdot 5 = 24$. Otra solución podría haber sido 8 0 0, pero como las estanterías más largas son más baratas la solución 3 3 0 es mejor.
$L = 29, l_1 = 3, l_2 = 9$	0 3 2	0 unidades de l_1 , 3 unidades de l_2 y quedan 2 unidades de medida sin estantería. $0 \cdot 3 + 3 \cdot 9 + 2 = 29$
$L = 24, l_1 = 4, l_2 = 7$	6 0 0	Con 6 unidades de l_1 dejamos la pared cubierta. $6 \cdot 4 + 0 \cdot 7 + 0 = 24$

Se pide:

- a) Diseñar una solución eficiente mediante el esquema voraz.
- b) Hacer una estimación del coste en tiempo de la solución.

Ej. 2 — (2,5 puntos) Dada una vara que mide n centímetros y un vector que indica los precios de todas las varas de tamaño menor o igual que n , determinar cuál es el máximo valor que podemos conseguir cortando la vara y vendiendo los trozos obtenidos.

Ejemplos:

- Si la longitud es 8 y los precios son:

longitud	1	2	3	4	5	6	7	8
precio	1	5	8	9	10	17	17	20

El máximo valor que podemos obtener es de 22 (cortando la vara en dos piezas de longitud 2 y 6).

■ Si la longitud es 8 y los precios son:

longitud	1	2	3	4	5	6	7	8
precio	3	5	8	9	10	17	17	20

El máximo valor que podemos obtener es de 24 (cortando la vara en 8 piezas de longitud 1).

Se pide:

- Si $\text{corteVara}(n)$ es el mejor beneficio que podemos obtener con una vara de longitud n , escribir la ecuación en recurrencias que permite calcular este valor.
- Diseñar un algoritmo que resuelva el problema del cálculo de dicho valor utilizando las ecuaciones recursivas.
- Analizar la eficiencia de la solución propuesta. No hace falta un análisis formal, basta con una idea genérica del coste en tiempo y espacio en función de n .
- Proponer una solución eficiente en tiempo para resolver el problema.

Ej. 3 — (2,5 puntos) En el *Cinelandia*, la última fila tiene asientos numerados del 1 al N . El asiento 1 está pegado a la pared, por lo que la única entrada a la fila es por el asiento N . Cuando alguien quiere llegar a algún asiento k , debe pasar por los asientos $N, N-1, \dots, k+1$, y cualquiera que ya esté sentado en esos asientos debe ponerse de pie para dejar pasar a la nueva persona (y luego se vuelven a sentar). Para la próxima proyección, tienes una lista $l[1..n]$ del orden en que llegarán los $n \leq N$ espectadores que tendrán que sentarse en la última fila, con sus números de asiento. Por ejemplo, la lista $[3, 1, 2, 5, 4, 9, 10]$ (7 espectadores) indica que la primera persona en llegar tendrá que sentarse en el asiento 3, la segunda en el asiento 1, etc. La primera persona en llegar tendrá que ponerse en pie dos veces (para dejar pasar a la segunda — con asiento 1 — y a la tercera — con asiento 2) y la cuarta persona en llegar (con asiento 5) tendrá que ponerse en pie una vez (para dejar pasar a la quinta — con asiento 4).

- Dada la lista $l[1..n]$, escribe un algoritmo (en pséudo-código) *eficiente** para calcular el número total de casos de ponerse de pie: la misma persona puede levantarse varias veces y todas esas cuentan por separado.
- Calcula el coste asintótico $O(\cdot)$ en tiempo del algoritmo considerando la longitud n de la lista en entrada.

*Observación: En este problema algoritmo *eficiente* quiere decir con un coste asintótico mejor que $O(n^2)$, donde n es la longitud de la lista.

Ej. 4 — (2,5 puntos) Hay que distribuir n miembros de un comité alrededor de una mesa de trabajo redonda con n sillas. Se dispone de información sobre la calidad de la relación entre los miembros: sea C la matriz (simétrica) de conflictos, $C[x, y]$ es un valor comprendido entre 0 (sintonía perfecta) y 10 (aversión profunda) que indica el nivel de conflicto entre x y y . Se quiere aplicar el esquema de ramificación y poda de mínimo coste para determinar una colocación de los miembros en la mesa que minimice el nivel de conflicto general. El conflicto general se calcula sumando los niveles de conflicto de los miembros sentados en posiciones adyacentes. Se pide definir:

- La representación de las soluciones y el árbol de búsqueda;

- b)** La función de coste $c(x)$;
- c)** La función de estimación del coste $\hat{c}(x)$;
- d)** La función de poda $\mathcal{U}(x)$.

ALGORITMIA BÁSICA

Grado en Ingeniería Informática y Doble Grado en Matemáticas-Ingeniería Informática

Examen de la primera convocatoria (19/05/2025)

- Resolver los ejercicios en **hojas separadas**. Todas las hojas deben llevar apellidos, nombre y número de orden.
- Si no da tiempo a resolver alguna parte, se tendrá en cuenta el indicar en *detalle* cómo se resolvería.
- Si algo no se especifica explícitamente, indicar las decisiones adoptadas (ej.: "como no se indica lo contrario, supongo que ...").
- **Comentar** los algoritmos adecuadamente.
- No se admitirán soluciones a lápiz ni con bolígrafo rojo.
- Duración máxima del examen: **3 horas**

Ej. 1 — (2,5 puntos) Dados dos vectores *llegadas* y *salidas* que representan los horarios de llegada y salida en una estación de trenes, respectivamente, necesitamos encontrar el mínimo número de andenes necesarios para que ningún tren tenga que esperar.

Ejemplo ₁	t_1	t_2	t_3	t_4	t_5	t_6
llegadas	9:00	9:40	9:50	11:00	15:00	18:00
salidas	9:10	12:00	11:20	11:30	19:00	20:00

Ejemplo ₂	t_1	t_2
llegadas	1:00	5:00
salidas	3:00	7:00

Resultados:

Ejemplo₁. 3 andenes (t_2 , t_3 y t_4 llegan entre las 9:40 y las 11:00 y salen desde las 11:20).

Ejemplo₂. 1 anden

Desarrollar un algoritmo que implemente una solución eficiente para este problema.

Ej. 2 — (2,5 puntos) Dada una matriz triangular, encontrar el camino de suma mínima de arriba a abajo, desde la fila 1 y columna 1; en cada paso podemos movernos a las posiciones adyacentes de la fila siguiente, esto es, si estamos en la columna j de la fila i podemos ir a la columna j o la $j + 1$ de la fila $i + 1$.

Ejemplos:

M_1	1	2	3	4
1	2	0	0	0
2	3	7	0	0
3	8	5	6	0
4	6	1	9	3

M_2	1	2	3	4
1	3	0	0	0
2	6	9	0	0
3	8	7	1	0
4	9	6	8	2

Resultados:

Matriz M_1 : 11. El camino es $2 \rightarrow 3 \rightarrow 5 \rightarrow 1$, y la suma mínima es: $2 + 3 + 5 + 1 = 11$

Matriz M_2 : 15. El camino es $3 \rightarrow 9 \rightarrow 1 \rightarrow 2$, que suma: $3 + 9 + 1 + 2 = 15$.

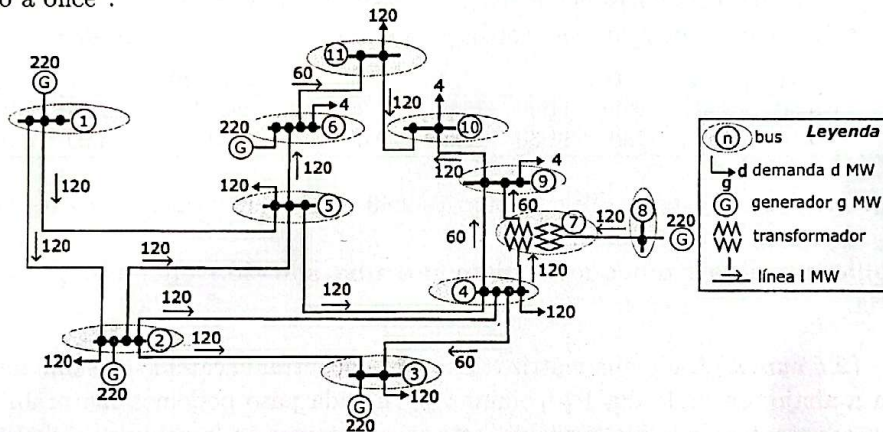
Resolver el problema, según los siguientes pasos: 1) Escribir la ecuación en recurrencias que representa el cálculo de la suma mínima. 2) Diseñar un algoritmo de programación dinámica que resuelva el problema del cálculo de la suma mínima en una matriz dada. 3) Analizar la eficiencia de la solución propuesta. No hace falta un análisis formal, basta con una idea genérica del coste. 4) Proponer un algoritmo que implemente una solución eficiente para resolver el problema.

Ej. 3 — (2,5 puntos) La elección del papa se realiza en gran secreto en cónclaves que acaban con “fumata”: las papeletas electorales se queman en un horno especial para evitar divulgación de información fuera del cónclave. La elección tiene éxito si hay mayoría de dos tercios de los votos en el cónclave: en este caso, el humo sale blanco. De lo contrario, el humo sale negro y se vuelve a repetir el proceso de elección. En estas elecciones hubo n cardenales electores y todos votaron. No se sabe exactamente cuantos candidatos “papables” hubo. Con el objetivo de agilizar el proceso electoral, el Vaticano te ha contratado para implementar un algoritmo *eficiente*¹ que determine si hay mayoría. Dados n votos, el algoritmo tiene que escribir por pantalla “blanco” (hay mayoría) o “negro” (no hay mayoría). No tienes acceso al contenido de las papeletas, la única función que puedes utilizar es:

```
//Pre: true
//Post: iguales(papeletaA,papeletaB) = (papeletaA == papeletaB)
void iguales(tipoVoto papeletaA, tipoVoto papeletaB);
```

Especifica el algoritmo y explica su coste asintótico en tiempo.

Ej. 4 — (2,5 puntos) Una red eléctrica incluye múltiples componentes como líneas de transmisión, transformadores, generadores, etc. Un *bus* es un nodo de la red donde convergen múltiples componentes. Considera la red en la figura con once buses, numerados de uno a once².



Un bus puede tener una demanda de energía de d MW (megavatios); estar conectado a un generador de potencia g MW, e incluir transformadores que pueden cambiar el nivel de voltaje —por ej., el bus 7 (transforman de 120 a 60MW). Una línea conecta dos buses: cada línea tiene asociada una dirección del flujo de energía “→” y una potencia máxima de transporte de l MW. Si el flujo de energía en una línea alcanza su capacidad máxima puede haber problemas de inestabilidad de la red.

Formaliza el problema para: 1) determinar si la red en la figura puede satisfacer la demanda total de 612 MW (la demanda total es la suma de todas las demandas); 2) identificar las líneas que pueden causar inestabilidad en la red. Resuelve el problema y justifica las respuestas.

¹Puedes considerar eficiente un algoritmo con coste en tiempo asintótico menor que $O(n^2)$, al menos en el caso promedio.

²La red y las potencias anotadas son adaptadas al problema de examen y no reflejan datos reales.